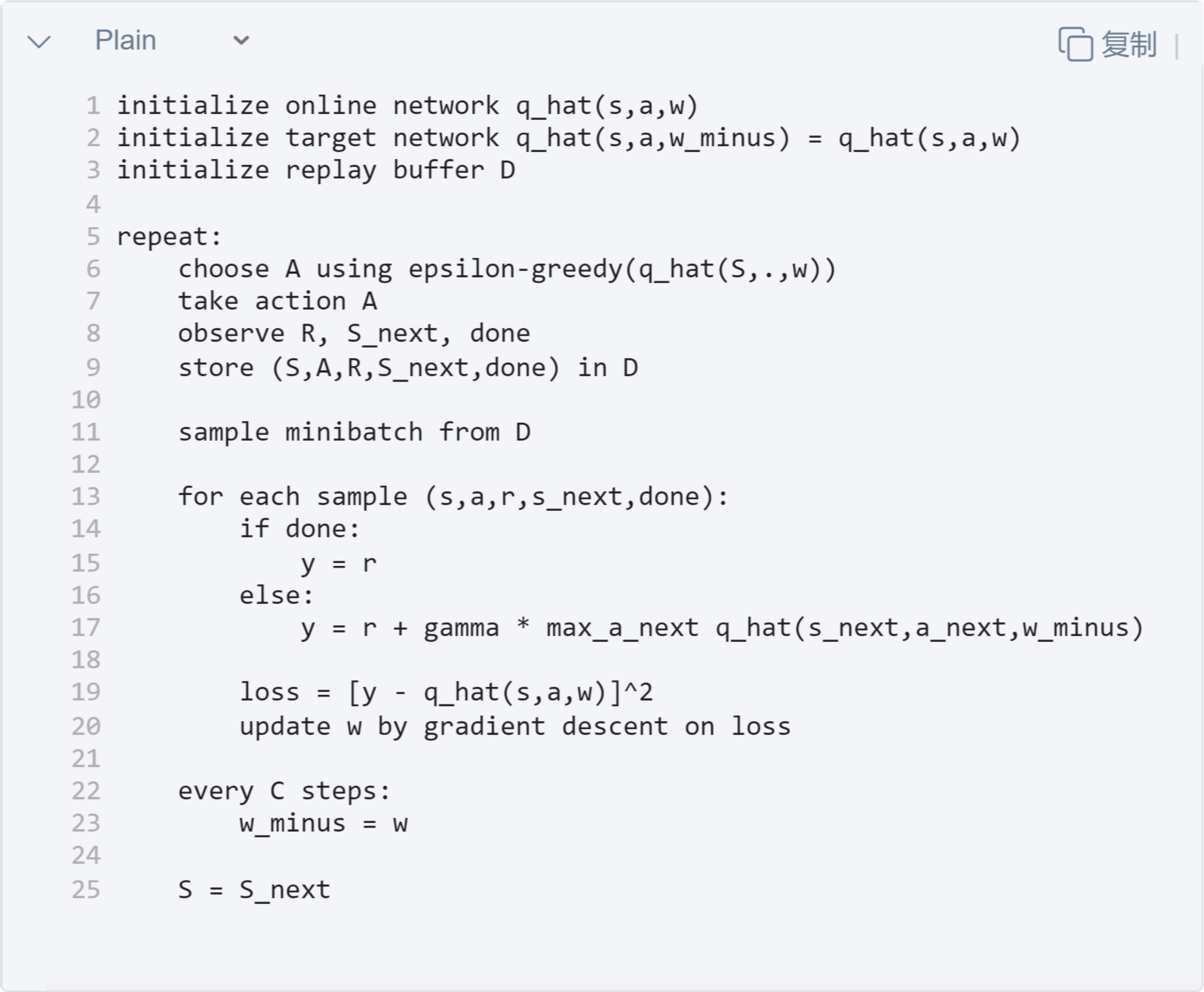
Value Function Model

：DQN（Deep Q-Network）



```
initialize online network q_hat(s,a,w)
initialize target network q_hat(s,a,w_minus) = q_hat(s,a,w)
initialize replay buffer D

repeat:
    choose A using epsilon-greedy(q_hat(S,.,w))
    take action A
    observe R, S_next, done
    store (S,A,R,S_next,done) in D

    sample minibatch from D

    for each sample (s,a,r,s_next,done):
        if done:
            y = r
        else:
            y = r + gamma * max_a_next q_hat(s_next,a_next,w_minus)

        loss = [y - q_hat(s,a,w)]^2
        update w by gradient descent on loss

    every C steps:
        w_minus = w

    S = S_next
```

当前状态 S
↓
epsilon-greedy 选动作 A
↓
执行动作
↓
得到 (R, S_next)
↓
存进 replay buffer
↓
随机抽 batch
↓
用 target network 计算 target y
↓
当前网络预测 Q(s,a)
↓
计算 loss
↓
梯度下降更新 w
↓
周期性同步 target network

DQN是off-policy

因为：amaxQ(s′,a)

不依赖实际下一动作。

即使行为策略是：

epsilon-greedy

target 仍然假设：

下一步选最大 Q 动作

所以：产生数据的策略 ≠ 更新 target 的策略

因此是：off-policy

$Q(s,a) \approx q^(s,a;w)$

Loss

代码：
loss = [y - q_hat(s,a,w)]^2

这其实就是监督学习。

当前网络预测：q^(s,a;w)

Q-learning target：y

然后最小化：(y−q^)2

也就是：让当前 Q 网络输出越来越接近 TD target。

target network 是两个模型一起学吗？不是。

这点很重要。

DQN 不是同时训练两个独立网络。

它是：
online network w：真正被梯度下降更新
target network w-：online network 的延迟拷贝

target network 不直接通过 loss 训练。
它只是每隔一段时间复制 online network：w−←w

所以它不是两个学生一起学习，而是：
一个学生在学习；另一个是他前一段时间的稳定版本，用来当临时答案参考。

Q-learning + 神经网络 + experience replay + target network

核心公式小变化

Q-learning target：

$y = r + \gamma a' max Q(s', a')$

DQN 只是把 Q 表换成神经网络：

$y = r + \gamma a' max q^(s', a'; w-)$

注意：
当前网络参数：w
target network 参数：w−
更新目标是：$[y - q^(s,a;w)]\mathbf{2}$

也就是：让当前网络预测的 Q 值，逼近 Q-learning target。

experience replay

你的代码里： initialize replay buffer D

这是经验回放池。每一步交互都会存：(S,A,R,S_next,done)

也就是 transition。

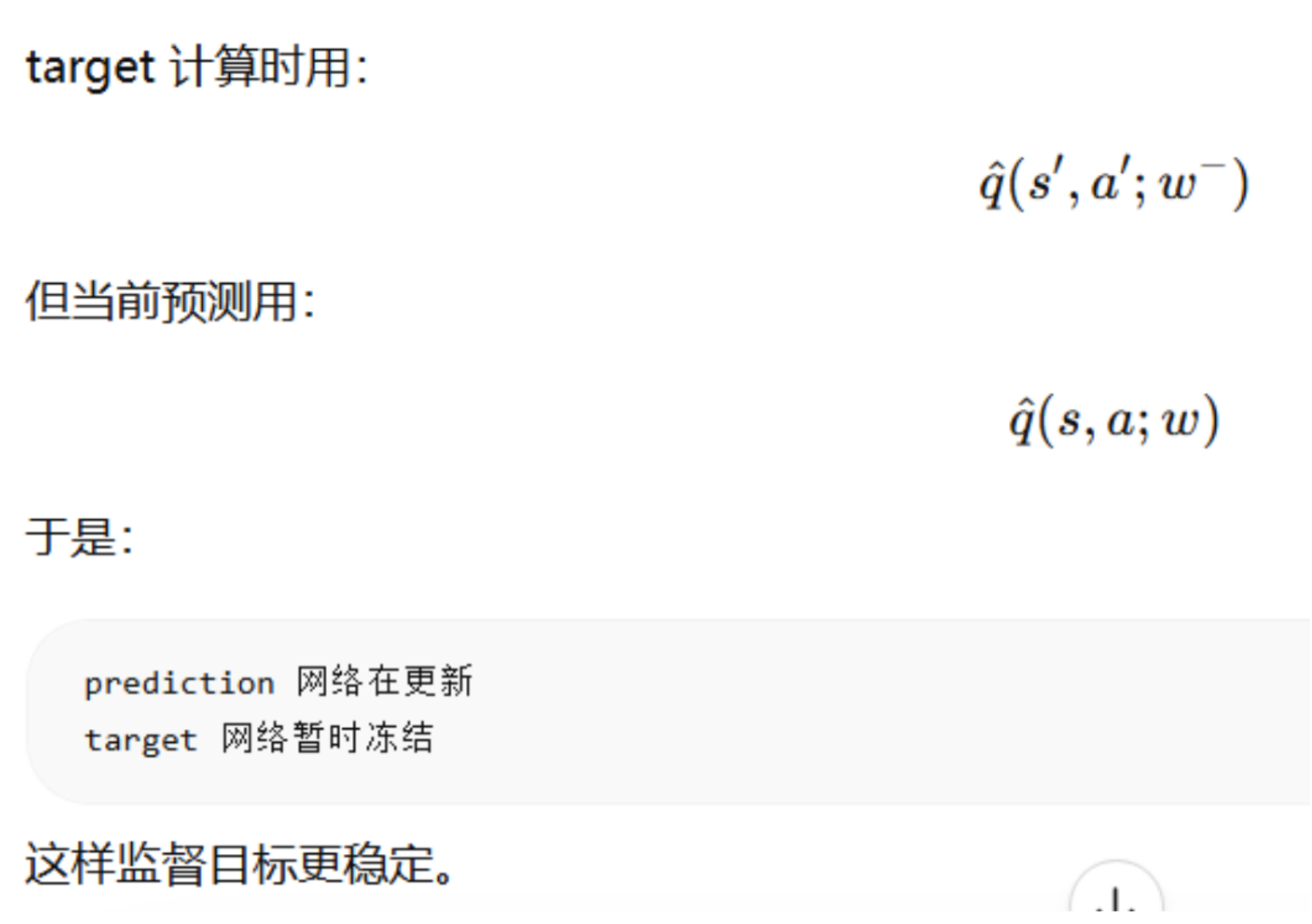
为什么需要target network

如果直接用同一个网络：
$y = r + \gamma a' max q^(s', a'; w)$

然后又用这个 y 去更新：
$q^(s,a;w)$

target 和 prediction 都来自同一个不断变化的网络。非常容易发散。

所以引入一个慢网络



target 计算时用：

$$\hat{q}(s',a';w^-)$$

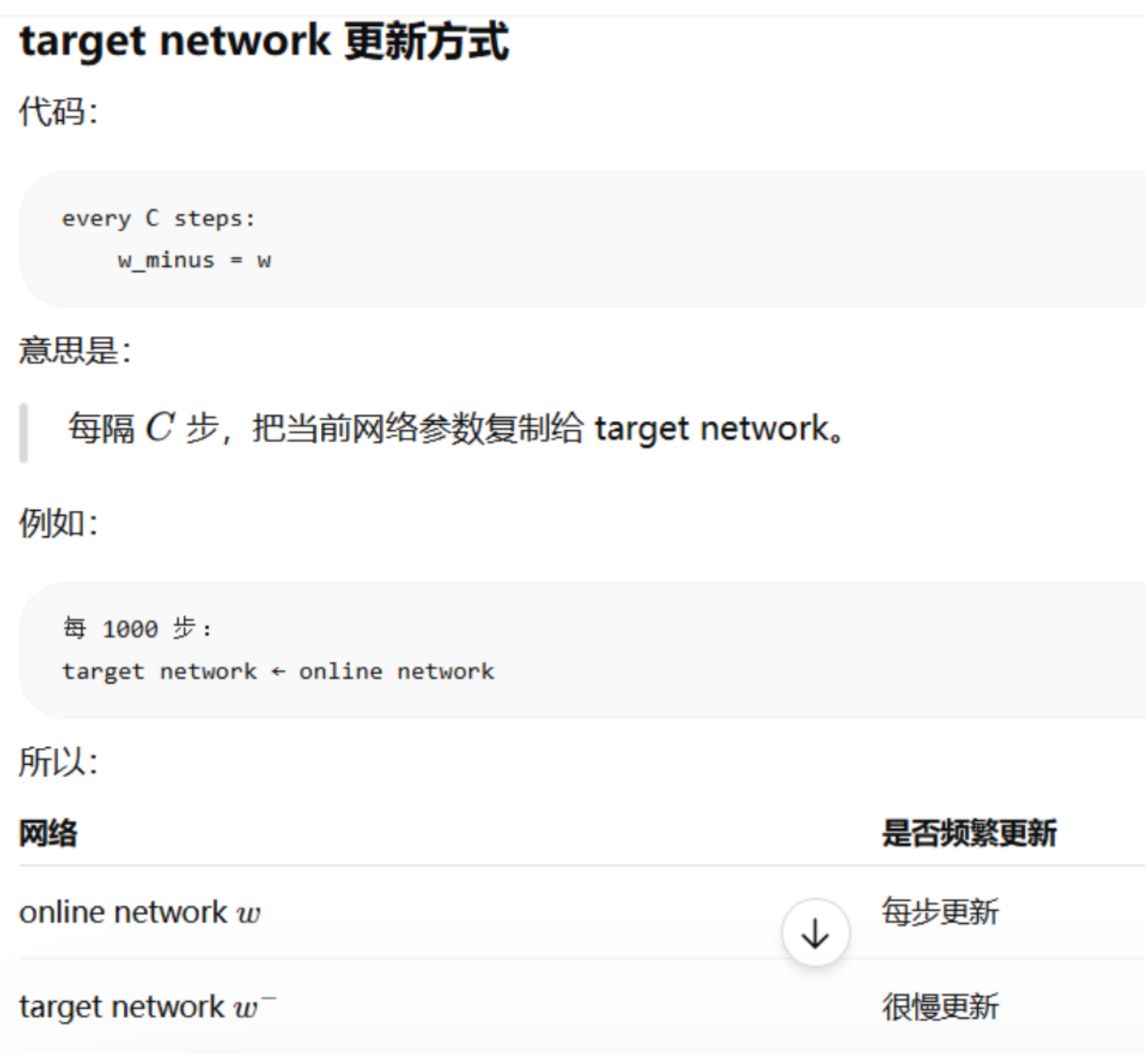
但当前预测用：

$$\hat{q}(s,a;w)$$

于是：

```
prediction 网络在更新
target 网络暂时冻结
```

这样监督目标更稳定。



target network 更新方式

代码：

```
every C steps:
    w_minus = w
```

意思是：

> 每隔 C 步，把当前网络参数复制给 target network。

例如：

```
每 1000 步：
target network ← online network
```

所以：

网络	是否频繁更新
online network w	每步更新
target network w^-	很慢更新